



HEILBRONN UNIVERSITY
OF APPLIED SCIENCES

Deep Learning (285652)

How Sequence Length Reveals Architecture Limits

Wagner Nils* Wenzel Nick†

May 7, 2026

Submitted to Carsten Lanquillon

*212405, nwagner1@stud.hs-heilbronn.de

†207678, nwenzel@stud.hs-heilbronn.de

1 Abstract

This report investigates how TextCNN, vanilla RNN, and LSTM architectures differ in their ability to model short-range and long-range dependencies in text, and how maximum sequence length affects sentiment classification. Using the IMDB Large Movie Review Dataset, we trained all models under the same preprocessing, vocabulary, embedding size, optimizer, and train/test split while varying the maximum input length from 32 to 512 tokens. The results show two effects of sequence length. Short inputs create content incompleteness, limiting all architectures. Longer inputs reveal architectural differences: TextCNN improves steadily, suggesting that local sentiment patterns remain highly predictive when more review content is available. The vanilla RNN plateaus around 128 tokens and collapses near chance accuracy at longer lengths, consistent with long-range optimization difficulties. The LSTM improves up to 128 tokens but also degrades at longer lengths in this implementation. Overall, TextCNN is the strongest model for this production-style sentiment task because it combines the best accuracy with stable training behavior and robustness to input length.

2 Introduction

Text classification requires models to transform variable-length sequences into fixed-size decisions. This is difficult because sentiment evidence may appear as short local phrases, such as “not worth watching”, or may depend on longer discourse structure, such as an initially positive description followed by a negative conclusion. The research question of this report is: *How do TextCNN, vanilla RNN, and LSTM differ in their ability to model short-range versus long-range dependencies in text, and what role does sequence length play?*

The three architectures provide contrasting inductive biases. TextCNN applies convolutional filters over local n -gram windows and aggregates the strongest activations through max pooling, which makes it well suited to local pattern detection [1]. A vanilla RNN processes tokens sequentially and can, in principle, encode earlier tokens into its hidden state, but it is known to suffer from vanishing and exploding gradients over long sequences [3]. LSTM networks address this limitation using gated memory cells that provide a more stable path for information and gradients [2]. Sequence length is therefore not merely a preprocessing detail: short lengths remove content, while long lengths stress a model’s ability to preserve and exploit information over many recurrent steps.

We study this question on the IMDB Large Movie Review Dataset [4], using sequence length variation to separate content incompleteness from architectural limitations in exploiting longer contexts.

3 Methods

The experiments use the IMDB Large Movie Review Dataset, a balanced binary sentiment classification benchmark with 25,000 training and 25,000 test reviews [4]. All models use the standard IMDB train/test split and the same preprocessing pipeline: lowercasing, HTML line break removal, replacement of non-alphanumeric characters, whitespace tokenization, integer encoding, and padding or truncation to a fixed maximum length. The vocabulary is built only from the training split, with a maximum size of 20,000 tokens and a minimum frequency of 2.

We compare TextCNN, vanilla RNN, and LSTM under identical training conditions. TextCNN uses convolutional filter sizes 3, 4, and 5 with 100 filters each, followed by max pooling and a fully connected layer. The RNN and LSTM use a hidden dimension of 128 and classify from the final hidden state. All models use embedding dimension 100, batch size 64, Adam with learning rate 10^{-3} , dropout 0.5, random seed 42, and 10 training epochs.

The maximum sequence lengths are 32, 64, 128, 256, and 512 tokens. This variation tests both content incompleteness and architectural limits: only 0.3%, 4.8%, 24.1%, 70.0%, and 92.1% of training reviews are fully contained at these respective lengths. Thus, short inputs often remove relevant content, while longer inputs provide more information but also require models to handle longer computational dependencies.

4 Results

4.1 Predictive performance

Table 1 reports test accuracy and macro F1 across sequence lengths. TextCNN improves monotonically and obtains the best overall result at 512 tokens. The vanilla RNN improves only slightly up to 128 tokens and then collapses near chance level at 256 and 512 tokens. LSTM improves from 32 to 128 tokens but also degrades at longer lengths. Macro F1 follows the same trend as accuracy, with especially strong degradation for the recurrent models at 512 tokens.

Metric	Architecture	32	64	128	256	512
Accuracy	TextCNN	0.7009	0.7483	0.8057	0.8506	0.8710
Accuracy	Vanilla RNN	0.6848	0.7027	0.7036	0.5000	0.5071
Accuracy	LSTM	0.7054	0.7451	0.8052	0.6990	0.5047
Macro F1	TextCNN	0.7009	0.7481	0.8057	0.8506	0.8710
Macro F1	Vanilla RNN	0.6848	0.7024	0.7026	0.5000	0.4358
Macro F1	LSTM	0.7043	0.7451	0.8052	0.6973	0.3753

Table 1: Test accuracy and macro F1 by architecture and maximum sequence length.

Figure 1 visualizes these trends. TextCNN continues to benefit from additional input, whereas the recurrent models do not reliably exploit longer sequences.

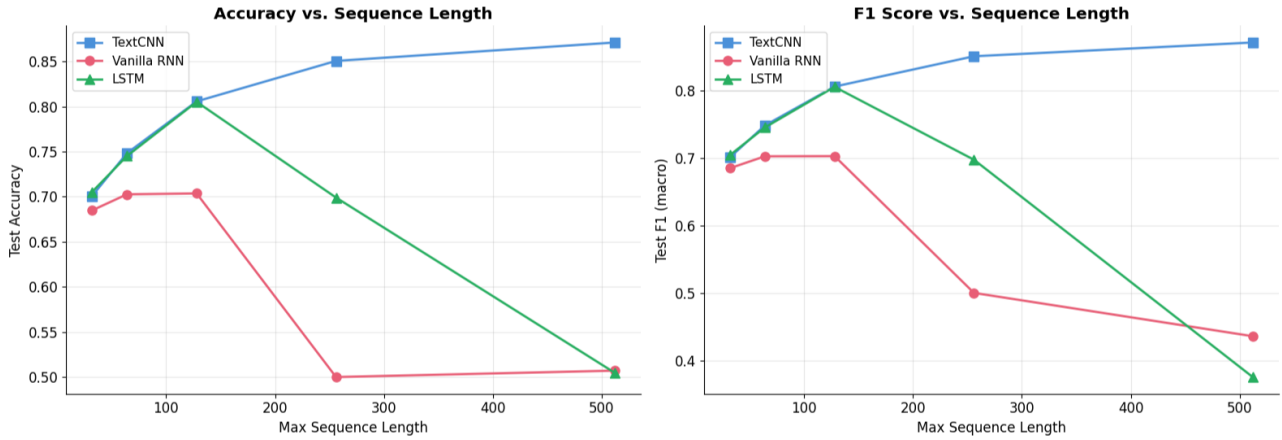


Figure 1: Architecture Comparison Across Sequence Lengths

4.2 Training time and model size

Table 2 and Figure 2 report the average training time per epoch for the CUDA run. The GPU measurements show only moderate runtime differences between architectures and sequence lengths. Although RNNs and LSTMs are sequential over time, CUDA accelerates the dense matrix operations inside each recurrent step by parallelizing them across the batch and hidden dimensions.

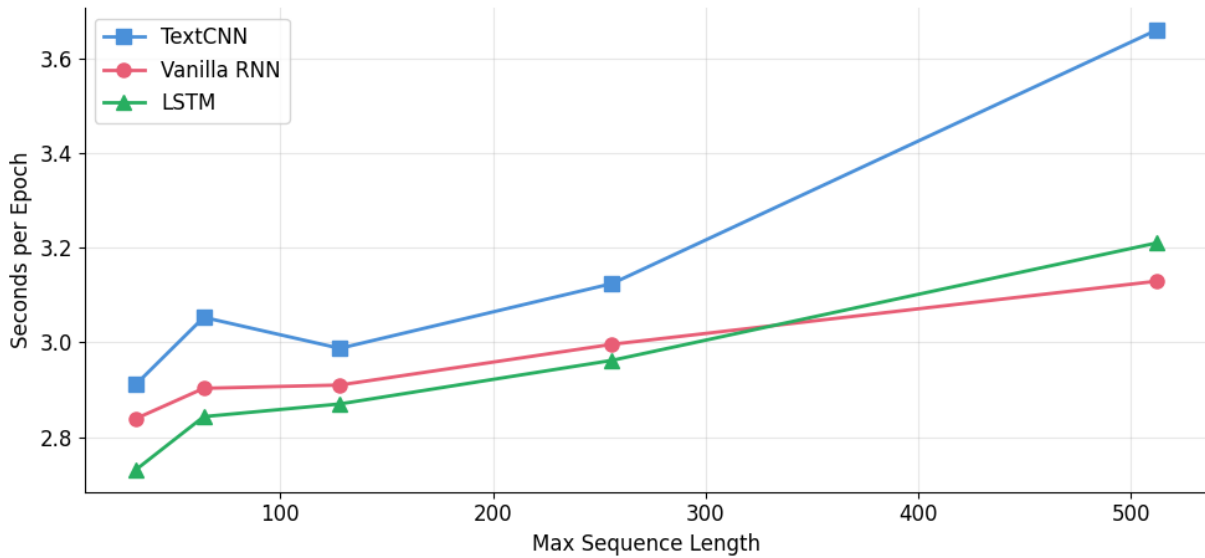


Figure 2: CUDA training time per epoch as a function of maximum sequence length. In the measured GPU setting, all three architectures show only moderate increases in training time.

Architecture	32	64	128	256	512
TextCNN	2.9	3.1	3.0	3.1	3.7
Vanilla RNN	2.8	2.9	2.9	3.0	3.1
LSTM	2.7	2.9	2.9	3.0	3.2

Table 2: CUDA training time in seconds per epoch by architecture and maximum sequence length.

Figure 3 shows the CPU run. In contrast to CUDA, training time increases much more strongly with sequence length, especially for the recurrent models. CPUs cannot exploit the batch-wise and hidden-dimension parallelism of recurrent matrix multiplications to the same extent as GPUs, so the repeated sequential computations of RNNs and LSTMs become increasingly expensive. TextCNN scales more favorably because convolutions can be applied across token positions.

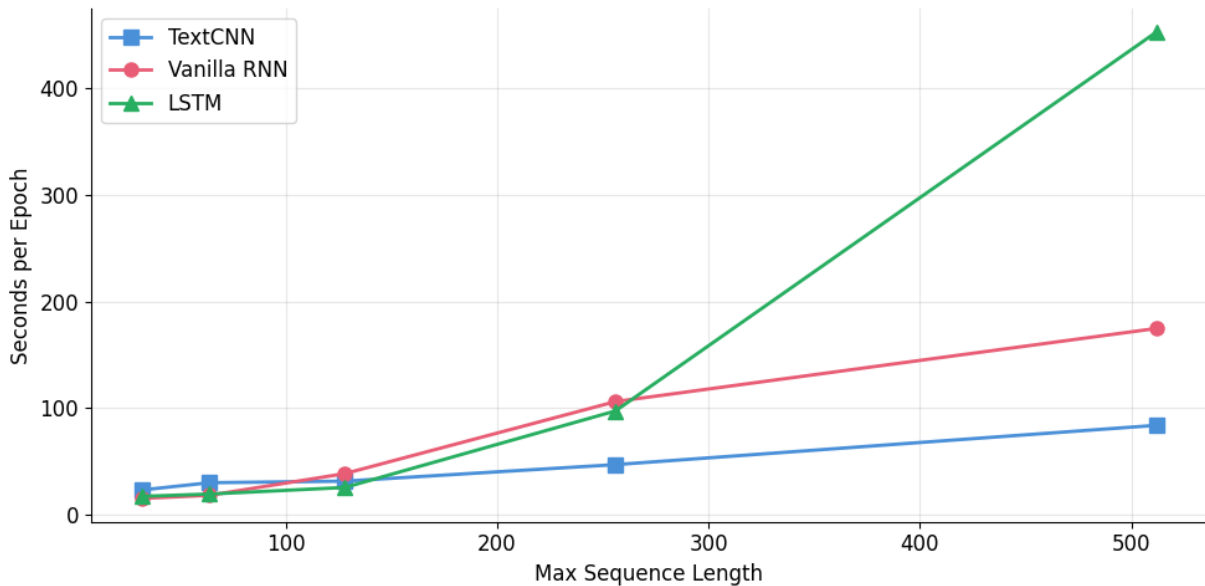


Figure 3: CPU training time per epoch as a function of maximum sequence length. Recurrent models show a much stronger increase at longer sequence lengths than TextCNN, especially the LSTM at 512 tokens.

Table 3 reports the parameter counts for the three architectures. TextCNN and LSTM have similar numbers of trainable parameters, while the vanilla RNN is smaller. Therefore, the stronger TextCNN performance cannot simply be explained by a larger model size.

Architecture	Total parameters	Parameters excluding embeddings
TextCNN	2,120,902	120,902
Vanilla RNN	2,029,698	29,698
LSTM	2,118,018	118,018

Table 3: Parameter counts by architecture.

4.3 Confusion matrices and error analysis

The notebook selected sequence length 512 for error analysis because it produced the largest accuracy gap between architectures. Figure 4 shows the confusion matrices at this length.

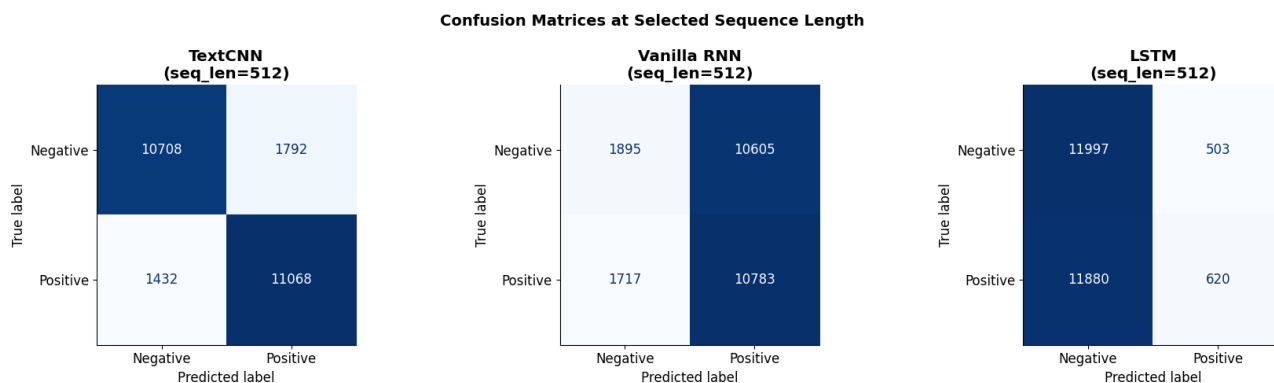


Figure 4: Confusion matrices at maximum sequence length 512. The sequence length was selected because it produced the largest observed accuracy gap between architectures.

Table 4 summarizes shortened representative examples from the error analysis. To keep the report concise, the table shows two examples per category; the full set of 3–5 examples per category is provided in the accompanying notebook. The examples illustrate three disagreement patterns: cases where LSTM benefits from gated memory compared with the vanilla RNN, cases where TextCNN succeeds through local sentiment cues, and cases where all models fail on ambiguous or misleading reviews.

Category	Review excerpt	True	TextCNN	RNN	LSTM
LSTM correct, RNN wrong	“I love sci-fi and am willing to put up with a lot... I tried to like this, I really did...”	Neg.	Neg.	Pos.	Neg.
LSTM correct, RNN wrong	“STAR RATING... Former New Orleans homicide cop Jack Robideaux...”	Neg.	Neg.	Pos.	Neg.
TextCNN correct, LSTM wrong	“That snarl... That scowl... The acts of random violence...”	Neg.	Neg.	Pos.	Pos.
TextCNN correct, LSTM wrong	“This is a hard film to rate. While it truly deserves its 3...”	Neg.	Neg.	Pos.	Pos.
All wrong	“My husband brought this home from the video store... a sorta silly, light, romantic...”	Neg.	Pos.	Pos.	Pos.
All wrong	“I watched Deadly Voyage because David Suchet was in it... with an expectation...”	Neg.	Pos.	Pos.	Pos.

Table 4: Representative error-analysis examples at sequence length 512. Excerpts are shortened from the notebook output.

5 Discussion

This study directly answers the research question: TextCNN, vanilla RNN, and LSTM differ mainly in how they use additional sequence length. TextCNN models short-range dependencies most effectively because its convolutional filters detect local sentiment patterns such as short negations or evaluative phrases. It does not explicitly model long-range order, but for IMDB sentiment this is not a major disadvantage because many decisive cues occur locally and can appear anywhere in the review. As sequence length increases, TextCNN receives more opportunities to find such evidence, explaining its steady improvement.

The vanilla RNN behaves differently because it must compress the whole prefix into a single hidden state. At short and medium lengths this is still sufficient, but at longer lengths the model has to carry information through many recurrent transitions. This makes earlier sentiment cues increasingly difficult to preserve and also makes optimization harder due to vanishing gradients [3]. The fact that the vanilla RNN plateaus around 128 tokens and then collapses at 256 and 512 tokens is therefore not simply a result of longer input. More tokens should provide more information. Instead, the model fails because its architecture cannot reliably exploit that additional information.

The LSTM was designed to reduce this problem through gated memory and a cell state that provides a more stable path for information and gradients [2]. This explains why it remains stronger than the vanilla RNN at 256 tokens. However, the LSTM does not maintain its performance at 512 tokens in this experiment. This does not necessarily contradict the gradient-highway theory; rather, it shows that gated memory alone is not sufficient under this implementation. Since the classifier uses only the final hidden state, relevant sentiment information can still be diluted across very long padded or truncated sequences. Without masking-aware packing, attention, pooling over hidden states, or pretrained embeddings, even an LSTM may fail to turn longer input into better predictions.

Sequence length therefore has two roles. At short lengths, it mainly controls content completeness. Truncating reviews to 32 or 64 tokens removes much of the original text, so low performance can be caused by missing evidence rather than by architecture alone. This is why the models are relatively close at 32 tokens. At longer lengths, however, content becomes less incomplete, so differences between the curves are more informative about architecture. The key finding is that TextCNN improves when more content is available, while the recurrent models fail to benefit reliably. This pattern indicates an architectural limitation rather than a content limitation, because the additional tokens are available to all models.

The hardware comparison adds a practical dimension. On CUDA, training times remain similar because GPUs accelerate dense matrix operations inside recurrent steps. On CPU, recurrent models become much slower at longer sequence lengths because their time-step dependency limits parallel execution. Thus, runtime conclusions depend on hardware, but the predictive conclusion remains unchanged: TextCNN is the most accurate and robust model in

this experiment.

For a production sentiment classification system under these conditions, TextCNN is the best choice. It achieves the strongest performance, benefits from longer inputs, has a moderate parameter count, and scales more favorably on CPU while remaining efficient on CUDA. The vanilla RNN should not be used for long reviews because it is both less accurate and architecturally fragile. The LSTM is theoretically better suited to long-range dependencies, but in this setup its additional complexity is not justified by the results. If a task required more subtle discourse-level reasoning, sarcasm, or long-range negation, a stronger sequence model with masking, attention, or a pretrained Transformer encoder would be a more appropriate next step [5].

This study has several limitations. First, the results are based on a relatively simple architecture setup using randomly initialized embeddings rather than pretrained or contextualized representations. Second, the recurrent models classify only from the final hidden state and do not use masking-aware sequence packing, attention, or pooling over hidden states, which likely disadvantages them on long padded sequences. Third, the experiment follows the required fair-comparison setup with a fixed random seed, but it does not include repeated runs across multiple seeds; therefore, the stability of the exact numerical results cannot be estimated. Finally, the conclusions are specific to binary IMDB sentiment classification, where local sentiment cues are often highly predictive. Tasks requiring precise long-range reasoning may produce different architectural rankings.

References

- [1] Y. Kim, *Convolutional Neural Networks for Sentence Classification*, Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1746–1751, 2014.
- [2] S. Hochreiter and J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] R. Pascanu, T. Mikolov, and Y. Bengio, *On the Difficulty of Training Recurrent Neural Networks*, Proceedings of the 30th International Conference on Machine Learning (ICML), pp. 1310–1318, 2013.
- [4] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, *Learning Word Vectors for Sentiment Analysis*, Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp. 142–150, 2011.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.